Transcript

Week 17: Deep Learning Optimisation

Video 1 – Deep Learning Optimisation

Training a model isn't magic—it's smart maths in motion. Optimisation is what helps your model go from clueless to clever, one step at a time. Let's see how it learns to level up.

In deep learning, optimisation means improving how well a model makes predictions. It does this by adjusting internal parameters—like weights and biases—to find the best possible combination. The goal is to minimise something called a *loss function*, which simply measures how far off the model's prediction is from the actual answer. By reducing this error, we help the model learn from its mistakes and get better at predicting new, unseen data.

To train a model effectively, we first need to measure how far off its predictions are from the actual answers. That's where the **loss function** comes in—it tells us how well or poorly the model is doing.
For regression tasks, we often use **Mean Squared Error**, which penalises large errors more heavily. In classification problems, we rely on **Cross-Entropy**, which compares predicted class probabilities with actual labels.
Another key factor is the **learning rate**. It controls how big each update step is during training. If the rate is too high, the model might overshoot the solution. If it's too low, training becomes slow. Finding the right pace is essential for smart learning.

Once we know how far off the predictions are, we need a way to improve them. That's where **optimisation algorithms** step in—they adjust the model's parameters to reduce the loss.
The most common method is **Gradient Descent**, which updates the parameters by moving in the opposite direction of the gradient. A faster version, called **Stochastic Gradient Descent**, updates the model using just one training example at a time. This speeds up learning but can be noisier.
Finally, there's **Adam**, which combines the benefits of Momentum and R-M-S-Prop. It's efficient, adaptive, and works well in a wide range of training scenarios.

The optimisation process of a neural network begins with initialisation, where parameters like weights and biases are randomly set to offer varied starting points. During the forward pass, input data flows through the network, producing predictions. Then in the loss calculation stage, we compare these predictions to actual outputs to determine how far off the model is. In the backward pass, or backpropagation, we calculate how each parameter affects the loss. This leads into the parameter update, where the model's parameters are fine-tuned using optimisation algorithms like gradient descent. These steps—forward pass through parameter update—are repeated multiple times to improve performance. Finally, a stopping criterion is applied, ending the training either after reaching a target loss level or completing a set number of training cycles, known as epochs.

During optimisation, models can run into several roadblocks. One major issue is **local minima**, where the algorithm gets stuck at a low point instead of finding the best possible solution. In deep networks, gradients can either **vanish**—becoming so small that updates barely happen—or **explode**, growing too large and making training unstable. Another key challenge is **hyperparameter tuning**. Choosing the wrong learning rate or batch size can slow progress

or lead to poor results. Understanding these issues is essential for training effective models.

Think your model's just being stubborn? Dive into the optimisation process and see if it's stuck in a local minimum, battling exploding gradients, or just needs smarter tuning. Time to troubleshoot like a pro!

Video 2 - Stochastic Gradient Descent (SGD)

What if your model could learn faster by making a few smart guesses? That's the power of Stochastic Gradient Descent—quick, bold updates that keep learning moving forward.

Stochastic Gradient Descent, or SGD, is a faster and more dynamic variation of the classic gradient descent algorithm used in machine learning. Gradient descent helps models learn by iteratively adjusting parameters to minimise the loss or cost function. In batch gradient descent, each update step is computed using the entire dataset, which can be slow and resource-intensive. In contrast, SGD updates the model using just one data point at a time, significantly speeding up the learning process—especially useful for large datasets—while still guiding the model toward optimal performance.

Stochastic Gradient Descent, or S-G-D, updates the model using just one data point at a time. Each time the model sees a new example, it calculates the gradient of the loss function and adjusts the parameters slightly to reduce the error. This makes the learning process faster and more flexible, especially with large datasets. The update rule is:
theta equals theta minus alpha times the gradient of the loss function J, with respect to theta, evaluated at data point x-i and its label y-i.
This simple approach helps the model improve steadily with each new example.

In Stochastic Gradient Descent, learning starts with a cost function—J of theta—which measures how far the model's predictions are from actual values. The goal is to reduce this cost and improve accuracy.

Unlike batch gradient descent, which processes the entire dataset at once, S-G-D updates the model one data point at a time. This makes the learning process quicker and more responsive, especially with large datasets.

The key update step is expressed as:
theta equals theta minus alpha times the gradient of J with respect to theta.
This means we adjust the parameters in the direction that reduces the cost. Each tiny step pushes the model closer to better performance.

Stochastic Gradient Descent is a method used to minimize a cost function—in this case, J of theta equals theta minus two squared. The goal is to find the optimal value of theta, which is 2. We begin with an initial guess of theta equals 5 and aim to reach the optimal value of theta equals two. In each iteration, we calculate the gradient of the cost function and update theta accordingly. This process continues, with each update moving theta closer to the minimum. Over time, the value converges toward the optimal solution through these repeated adjustments.

Stochastic Gradient Descent works well for large datasets because it updates parameters using individual data points, rather than the entire dataset. This makes it faster and more efficient in high-volume scenarios.
Another advantage is that its noisy updates can help the algorithm escape local minima, which are points that seem optimal but aren't the best overall solution.
However, this same randomness can also cause instability. So, careful tuning of the learning rate is essential to help the algorithm settle at the right point.

Momentum is a technique that helps Stochastic Gradient Descent, or S-G-D, move faster in the right direction and avoid getting stuck. It works by remembering past updates and using that memory, called momentum, to guide the current update. This approach reduces the impact of noise in the data and helps the model converge more smoothly.

Let's look at how this works in terms of formulas:
The velocity at time *t* is calculated as
v-t equals beta times v-(t minus 1) plus one minus beta times g-t.

The updated parameter is then
theta-t plus 1 equals theta-t minus eta times v-t,
where *theta* is the parameter, *eta* is the learning rate, and *g* is the gradient.

Momentum helps in two major ways: it speeds up convergence and helps avoid local minima, especially in rough terrain.

Traditional gradient descent can struggle with three key issues: oscillating gradients, learning rate sensitivity, and varying update scales. In high-dimensional spaces, gradients can fluctuate wildly, slowing convergence and causing instability. Choosing the right learning rate is also tricky—too high, and the model overshoots; too low, and it learns painfully slowly.
Additionally, not all parameters require the same size of update. Some benefit from bold moves, while others need tiny adjustments.

This is where **adaptive optimisation algorithms** come in. They help by adjusting the learning rate dynamically for each parameter, leading to smoother and more efficient learning.

Let's connect this to a few popular adaptive methods:

- **R-M-S-Prop** smooths out updates using a moving average of squared gradients, making training more stable.
- **Adam** combines the benefits of Momentum and R-M-S-Prop, leading to faster convergence.
- **AdaGrad** adjusts the learning rate based on how often each parameter is updated—providing fine-grained control.
- **AdaDelta** builds on AdaGrad by fixing its shrinking learning rate problem, ensuring the model doesn't slow down too quickly.

Together, these adaptive methods tackle the core challenges of gradient descent—making model training more stable, efficient, and effective.

RMSprop, or Root Mean Square Propagation, is an optimiser that adjusts the learning rate for each parameter individually. It does this by dividing the learning rate by an exponentially decaying average of the squared gradients. This helps control the size of updates and reduces oscillations during training.

The update formula is:
Theta at time t plus one equals theta at time t minus eta multiplied by the gradient at time t, divided by the square root of the expected value of the gradient squared plus epsilon.

Here, **theta** represents the parameter being updated, **eta** is the learning rate, **g-t** is the current gradient, **E of g-t squared** is the moving average of squared gradients, and **epsilon** is a small constant added to prevent division by zero.

RMSprop works well in dynamic environments where the optimal parameters may shift over time. It also helps stabilise training in the presence of noisy gradients, making it a popular choice for training recurrent neural networks and working with large datasets.

Let's look at Adam—short for Adaptive Moment Estimation.
This optimiser combines the strengths of RMSprop and Momentum to create a powerful update rule.
It adapts the learning rate for each parameter, improving both training speed and stability.

Adam works by calculating two moving averages during training.
The first moment, *m-t*, is the average of the gradients and helps determine the direction of the update.
The second moment, *v-t*, is the average of the squared gradients and measures the variability of those updates.

Here's how the update works.
First, we calculate the first moment:
m-t equals beta one times m-t minus one plus one minus beta one times g-t.
Then the second moment:
v-t equals beta two times v-t minus one plus one minus beta two times g-t squared.
Finally, we update the parameter using this rule:
theta-t plus one equals theta-t minus eta times m-t divided by the square root of v-t plus epsilon.

In this formula:
theta is the parameter being updated.
eta is the learning rate.
beta one and *beta two* are decay rates, commonly set to zero point nine and zero point nine nine nine.
epsilon is a small constant that prevents division by zero.

This combination allows Adam to train efficiently across different parameter scales, even in complex networks.

Let's compare Adam and RMSprop, two widely used adaptive optimisers.
Both methods improve training efficiency by adjusting learning rates based on gradient history.

Adam uses estimates of both the first moment, or the mean of past gradients, and the second moment, or the variance. This dual approach allows for more stable and adaptive learning, especially in complex models.

RMSprop, on the other hand, uses only the second moment to adjust learning rates. It's a simpler method that works particularly well for recurrent neural networks and when training data is less noisy.

While RMSprop is preferred for low-noise, sequential data, Adam offers greater flexibility across a broader range of tasks, especially those involving sparse gradients.

Let's take a look at four additional optimisers, each designed to tackle specific challenges during training.

We begin with **AdaGrad**, which adapts the learning rate based on how frequently each parameter is updated. This makes it especially useful for sparse datasets, where some features are rarely used but still important.

Building on this, **AdaDelta** addresses a limitation of AdaGrad. By removing the need for a fixed learning rate, it adapts the step size using a moving average of past gradients. This makes it a better fit for complex models over longer training periods.

Next, we have **SGD with Momentum**. It introduces a momentum term into basic stochastic gradient descent, which helps the optimiser move past local minima and improve convergence—especially when gradients are noisy or unstable.

Finally, **Nadam** combines Adam with Nesterov momentum. This approach looks ahead at future gradients before making updates, often resulting in faster and more stable convergence, particularly in deep networks.

In summary, these optimisers offer targeted solutions to common training challenges—giving you more control, flexibility, and performance across different modelling scenarios.

When choosing an optimiser, consider a few key factors.
For sparse datasets, both Adam and AdaGrad are good choices. They adapt learning rates based on how frequently features are updated, helping models converge more effectively.

If you're working with complex models, like convolutional or recurrent neural networks, Adam is a reliable option thanks to its momentum and adaptive learning features.

In terms of training time, Adam and RMSprop usually converge faster than standard methods. But if long-term performance is your goal, Stochastic Gradient Descent with momentum may deliver better results.

A simple strategy is to start with Adam. From there, use validation performance to fine-tune and find the best optimiser for your task.

Even with adaptive optimisers like Adam and RMSprop, it's still important to fine-tune the learning rate. Testing different starting values can help improve training performance and convergence.
Batch size also plays a key role. Larger batches often give more stable results, but they require

more memory and may slow things down. It's all about finding the right balance for your task. Lastly, always use regularisation techniques like dropout or weight decay. These help prevent overfitting and improve how well your model performs on new data.

Test it out—tune the learning rate, tweak the batch size, add regularisation, and watch your optimiser transform model performance.

Video 3 – Regularisation

Why do some models perform brilliantly during training but fail miserably in the real world?

Regularisation is a technique used to stop a model from overfitting the training data. Overfitting happens when the model memorises both important patterns and random noise, which reduces its performance on new, unseen data. By adding constraints to the loss function, regularisation improves generalisation. This means the model becomes better at making accurate predictions beyond just the training examples.

In machine learning, overfitting is a common challenge, especially with complex models that learn the fine details of training data, including noise. This leads to high variance, where the model performs well on the training set but struggles to generalise to new data. Regularisation offers a solution. By adding a penalty to the loss function, it reduces model complexity, helping the model focus on meaningful patterns and improving its generalisation.

Let's explore the key types of **regularisation** techniques that help prevent overfitting and improve generalisation in models.

We'll start with **L-1 regularisation**, also known as Lasso. This method adds the absolute values of the model's coefficients as a penalty. By doing so, it encourages the model to reduce some weights to zero, which helps simplify the model and even leads to feature selection.

Next is **L-2 regularisation**, or Ridge. Instead of using absolute values, it adds the squared values of the coefficients as penalty. This shrinks the weights but rarely brings them to zero, which helps retain all features while keeping the weights small and reducing overfitting.

Now, when we combine both L-1 and L-2 regularisation, we get **Elastic Net**. This approach strikes a balance between weight reduction and feature selection, benefiting from the strengths of both methods.

Another powerful technique is **Dropout**, widely used in deep learning. During training, it randomly disables certain neurons. This forces the model to avoid over-reliance on any one path and instead use multiple pathways, which improves generalisation.

Finally, we have **Early Stopping**. This technique keeps an eye on validation performance. If the error starts to increase, training is paused to prevent the model from overfitting.

Together, these techniques help models remain balanced and generalise better to new, unseen data.

L1 regularisation, also known as Lasso, adds a penalty based on the absolute value of each

model coefficient.
This modifies the loss function to: *Loss equals original loss plus lambda times the sum of absolute weights*.
Here, **lambda** controls how strong the penalty is.
L1 regularisation pushes some weights to exactly zero, which makes the model simpler and easier to interpret.
This sparsity makes it especially useful for feature selection, as it removes less relevant inputs from the model.

L2 regularisation, also known as Ridge, adds a penalty based on the magnitude of a model's coefficients.
In the loss function, this is written as: **loss equals original loss plus lambda times the sum of squared weights**.
Here, **lambda** controls how strong the penalty is.
By encouraging smaller weights, L2 regularisation helps the model generalise better and reduces the risk of overfitting.
It's especially helpful when features are highly correlated, because it balances weight values rather than removing them entirely.

Elastic Net Regularisation combines the strengths of both L1 and L2 techniques. In the loss function, it adds two penalties to the original loss: lambda one multiplied by the sum of the absolute values of the weights, and lambda two multiplied by the sum of the squared weights. These penalties help shrink some coefficients to zero while keeping others small. This makes the model simpler and more stable, especially when features are strongly correlated. It's a practical choice when you want the benefits of both feature selection and regularisation.

Dropout regularisation is a method used in neural networks to reduce overfitting. During training, it randomly sets some input units, or neurons, to zero. This prevents the model from depending too much on any single feature. Each neuron is dropped independently with a certain probability, often written as *p*. By doing this, the network learns to build multiple paths and redundant representations. The result is a more robust model that generalises better to new data, even when some inputs change or are missing.

Early stopping is a technique used to prevent overfitting during training. It works by monitoring the model's performance on validation data. If the validation loss stops improving or starts to rise, it signals that the model may begin to overfit. At this point, training is stopped. This helps preserve the model in its best state—accurate, efficient, and generalised, without wasting time on unnecessary training.

Test out early stopping in a model you've built—observe how it helps you avoid overfitting and lock in your best performance.

Video 4 - Batch Normalisation

What's the secret to training deep networks faster—without changing the model itself?

Batch Normalization is a technique used to speed up and stabilize neural network training. It works by normalizing the inputs of each layer, addressing the issue of internal covariate shift.

This shift happens when the distribution of activations changes during training, slowing down the learning process. By standardizing the inputs to each layer, Batch Normalization makes the network learn more efficiently and consistently, which helps the model generalize better to new data.

Batch Normalisation helps speed up training by standardising activations across mini-batches. This stabilises the internal representations inside deep neural networks. It reduces something called *internal covariate shift*, where the distribution of activations changes during training. As a result, training becomes faster and more reliable. It also improves the model's ability to generalise, which means it performs better on unseen data.

Batch Normalisation brings three major benefits to deep learning. First, it speeds up training by stabilising the inputs to each layer, helping the model converge faster. Second, it creates a more stable learning environment by reducing fluctuations in gradient values. Finally, it acts like a regulariser, limiting overfitting and helping the model perform better on new, unseen data.

Batch Normalisation works in three main steps. First, it calculates the mean and variance of activations across a mini-batch. Then, it normalises the values by subtracting the mean and dividing by the standard deviation. Finally, it applies scaling and shifting using learnable parameters gamma and beta. Together, these steps help stabilise learning, speed up training, and improve model performance.

Here's how batch normalisation is calculated.
We take the activation of a given neuron, subtract the batch mean, and divide it by the square root of the batch variance plus a small constant.
This constant, epsilon, is used to maintain numerical stability.
The result is a standardised activation that improves training speed and model consistency.

Batch Normalisation doesn't just normalize activations—it also includes two learnable parameters that enhance flexibility. Gamma adjusts the scale of the output, letting the model learn the right activation range. Beta shifts the output, helping the model set the most effective activation mean. Together, they allow the model to retain the benefits of normalization while adapting to the data's original distribution when needed.

Batch Normalisation improves model performance in three important ways. First, it speeds up convergence by reducing internal covariate shift. This means the model can train in fewer steps. Second, it helps prevent overfitting by acting as a regulariser, improving the model's ability to generalise to new data. And third, it allows the use of larger learning rates, making training both faster and more effective.

Batch Normalisation works best in deep neural networks, especially when there are many layers. It's also helpful when a model is struggling with slow convergence or training instability. By normalising the inputs, training becomes faster and more stable. Another key benefit is its support for higher learning rates, which can speed up training, provided the process is stabilised during optimisation.

Think your model's training is too slow or unstable? Try applying batch normalisation and observe how it affects convergence and performance.

Video 5 – Advanced Regularisation Techniques- Demo

Welcome to this session on advanced regularisation techniques in deep learning. As our models grow more complex so does the risk of overfitting where the model performs well on training data but struggles to generate good response on unseen examples or struggles to generalise to unseen examples. While you may already be familiar with techniques like L1 L2 regularisation dropout and early stopping today we'll go a step further.

In the next section we will explore modern regularisation strategies used in real world deep learning applications covering techniques like data augmentation, label smoothing, gradient clipping and more. These tools can significantly improve model stability generalisation and training efficiency making them critical additions to your deep learning toolkit. Remember these techniques we try when the other techniques that we have discussed fail to solve our problem that's what generally we do.

So let's dive in let's begin by understanding why we even need advanced regularisation techniques. First complex data challenges traditional methods like L2 regularisation and dropout may work well on small or clean data sets but they often fall short when working with the high-resolution images natural language data or large-scale time series. These types of data introduce a level of complexity that basic techniques just can't handle well enough.

Second, we hit what's called performance ceiling. Basic regularisation can prevent overfitting to some extent but it often can't help the model generalise beyond a point to push past that ceiling especially in competitive or production level settings we need more adoptive and robust techniques. And finally stability concerns deeper and more complex architectures like transformers GANs and very deep convolutional neural networks require more than dropout and weight decay.

They need specialised strategies to ensure stable training and convergence. Now that we understand why going beyond basic regularisation is essential let's take a quick look at the techniques we will explore in this session. First is data augmentation a powerful data level technique where we apply transformations like rotation, flipping, cropping or colour shifts to artificially increase data set diversity.

Next is label smoothing which modifies target labels slightly to avoid over confidence in predictions. This subtle trick can make your models more robust especially in classification problems. Third we will dive into gradient clipping and constraints.

These help control gradient explosions and weight magnitudes especially critical when training deep or recurrent networks. And finally we'll finally we'll explore advanced stochastic methods like mix up cut mix and drop connect. These introduce control randomness into training improving generalisation and reducing overfitting.

Each of these techniques plays a unique role in enhancing model performance and together they offer a powerful toolkit for any deep learning practitioner. Now remember one thing some of these techniques we are simply introducing to you probably in our examples that we are dealing later they might not be required. We might use data augmentation in some cases label smoothing but beyond that probably in our examples we don't need it.

So this is more of an introduction session to some of the more advanced regularisation

techniques. Some of them we will be using later some of them is just for your intro purpose. Data augmentation.

Data augmentation is one of the most effective and widely used regularisation techniques in computer vision. What does it do? It creates new training samples by applying random transformations to the existing data especially used with the images like flipping rotating cropping or changing colours. These changes don't alter the meaning of the image but provide more variety for the model to learn from.

For example you can take an image of a dog and flip it and it becomes a new example for the model and that way we can augment data we can create more training data for our model to learn from. We can rotate the image a bit we can crop it and they all become new training data for the model. Similarly in speech recognition problems we can add some background noise to the audio files to create new training data for models to learn.

You can add traffic noise some music or probably how a marketplace is buzzing with sound all this goes to the background of an audio file and it creates more training examples for you. So the same data is getting augmented by adding some noise in case of audio or doing some transformation in the case of images and we create more data points. The question is why does it work? Because it teaches the model to become invariant to small changes that don't affect the label.

So for example a cat is still a cat even if the image is flipped or slightly rotated. This not only increases the effective data set size but also improve the model's generalisation ability. Now where is the most where is it most useful? Primarily in computer vision tasks like I mentioned image classification especially when collecting new label data is expensive or impractical.

If you have a small data set this can make huge difference in performance. Think of data augmentation is like making copies of a photo with different filters or angles. You don't need new photos you just edit the old ones.

This helps the model become more flexible and reduces its chances of memorising the training data. Label smoothing. Label smoothing is a simple but powerful regularisation technique especially useful in classification problems.

What's the concept? Instead of using hard labels like 1 0 0 this is an example of a classification problem where we have three classes and when we say 1 0 0 so if I'm classifying something as a b or c when we write 1 0 0 it means the object is a if I write 0 1 0 means object is b this is just the way we write it. So instead of using hard labels like 1 0 0 we slightly soften them into something like 0.9 0.5 and 0 point sorry 0.9 0.05 and 0.05. This introduces a small amount of uncertainty into the learning process making the model less confident in its predictions. But why do we do this? Models that are too confident tend to overfit and may perform poorly on unseen data.

Label smoothing encourages better generalisation and prevents the model from memorising noisy or incorrect labels. The formula shown here applies a smoothing factor denoted by epsilon and distributes some of the probability across all classes where k is the number of classes. In practise this helps the model become more calibrated and resilient especially useful in large classification problems like NLP or image classification.

One way of looking at would be that imagine you are teaching a child that an apple is a fruit and instead of saying this is 100 apple you say this is mostly an apple but there's a tiny chance it could be something else. That way the child doesn't become overconfident and often think before answering and is open to learning better with more examples. That's what label smoothening does.

It teaches the model not to be overly sure and helps it generate better responses or generalise better. Gradient clipping is a regularisation technique used to maintain stability during training especially in deep networks and recurrent models. Why do we need it? In some models especially RNNs and transformers which will be dealt in detail gradients can become very large during back propagation and gradients are the one that we are using to update the weights of the model.

Now if I take the simplest form of your gradient descent and I say the new weight would be equal to the old weight minus alpha times gradient. Now if this gradient becomes too small then training stops and it becomes too big then there is a bigger step that the model is taking or the weight updates are larger and my model might not converge and this phenomenon is called exploding gradient problem where your gradient becomes so big that there's a large update on your model and model's performance deteriorates. So this is called exploding gradients where parameter updates become too extreme and destabilise the training.

So what does gradient clipping do? It sets up a cap on maximum allowed gradient norm. If gradients exceed that threshold, they are scaled down proportionally. This ensures consistent and controlled weight updates leading to smoother convergence.

You can see the pytorch implementation over here just to give you an example where we clip gradients to a maximum of value as 1. This technique is especially helpful when training very deep networks or sequences with long dependencies where gradients may otherwise become unmanageable. Think of gradient clipping like putting a speed limit on a car. If the car tries to go too fast the limiter kicks in and slows it down.

Similarly if the gradient becomes too large clipping brings it back under control to avoid a crash in training. Now let's talk about two innovative data level regularisation techniques mix-up and cutmix. Mix-up works by blending two different images together and also blending their labels.

So if one image is cat and another is a dog the mixed image is a soft combination of both and the label is say 0.6 cat and 0.4 dog. This encourages the model to generalise across class boundaries. Cutmix on the other hand goes a step further.

It literally cuts a patch from one image and paste it onto another. The label is mixed in the proportion to the area of the patch. For example if 30% of the image is pasted region the label is adjusted accordingly.

These methods force model to learn relationships between classes rather than just memorising individual samples. The main benefit is that they help the model create smoother decision boundaries which leads to better performance on unseen or noisy data. Imagine taking two animal pictures say a cat and a dog and blending them like a smoothie.

The model then learns not just what a cat or a dog look like but how they differ and overlap

that's mix-up. Now imagine cutting a small window from one image and sticking it on another that's cutmix. It teaches the model to focus on important regions and not just memorise the whole picture.

These are relatively new concepts. It's just an introduction to you to understand what all is evolving in regularisation. Next drop connect is a more fine-grained alternative to the widely used drop out technique that you have gone through.

In drop out we randomly deactivate entire neurones during training but in drop connect we randomly deactivate individual connections between neurones instead. That means the neurone remains active but some of its links to other neurones are temporarily dropped. This adds another level of noise and forces the network to rely less on any specific connection leading to better generalisation.

So instead of switching off a neurone we are switching off some connection to the neurone. Like drop out drop connect uses a retention probability hyperparameter to control how many connections are retained. It's especially useful in fully connected layers of very deep networks where regular dropout may not be enough to prevent overfitting.

In summary while dropout removes entire neurones during training drop connect targets individual weights making it more precise and potentially more effective for certain architectures. If you want to understand in a simpler manner, think of neurones like cities and connections like roads. Dropout closes down whole cities temporarily but drop connect on the other hand just blocks a few roads.

The cities still function but they have to find alternative routes. This makes the network more flexible and less dependent on specific paths. Then we have max norm constraints.

Max norm constraints are another useful regularisation strategy especially when working with very deep or complex models. Mechanism wise this technique enforces an upper limit or ceiling on how large the weights in a network can grow. After each training step if a weight vector exceeds this limit, it's scaled back to stay within the defined bound.

In keras you can apply this using simple line like constraint is equal to max norm and then 3 that's what you can use in keras. In pytorch you would manually clip weights after each optimiser step. There you need to write custom code for it.

Why is this useful? Because it prevents uncontrolled growth of weights which can destabilise training or make the model overly sensitive by keeping weights magnitude in check. Max norm helps maintain stable learning dynamics especially in deep networks where optimisation can otherwise get chaotic. Imagine to understand in a better manner imagine you are blowing up balloons.

Max norm is like saying no balloon can grow bigger than a tennis ball. If one gets too big you let some air out that way all your balloons stay manageable and your model doesn't blow up during training. Now let's look at this comparison here.

Let's close with a quick side-by-side comparison of all the techniques we explored. Each of these regularisation methods operates at a different level of training process. Some modify the input data, some affect labels or gradients and other intervene in model architecture or

weight behaviour.

Data augmentation and mix up cut mix work at the data level improving robustness and helping the model generalise better. Label smoothing acts as a label level at a label level and is especially powerful in classification tasks to reduce model over confidence. Gradient clipping addresses exploding gradient and is most useful in models with long dependencies like RNNs or transformers.

Drop connect works at the architectural level and helps reduce overfitting by randomly dropping connections rather than neurones. Max norm constraint and force control overweight growth keeping optimisation stable over time. This table not only helps summarise but also act as a quick reference guide for when and where to apply each technique.

As a wrap-up let's talk about how to choose the right regularisation techniques for your model. Start by assessing your problem. Think about the model architecture, size of your data set and the complexity of your task.

For example are you using a deep CNN or a transformer? Is your data noisy or limited? Identify specific failure patterns like over confidence or instability during training. Second don't hesitate to combine techniques. Many regularisation methods work better when used together.

For instance pairing data augmentation with label smoothing is a great strategy for image classification. Just like spices in cooking using a blend often yield better flavours or in our case performance. And finally validate impact.

Regularisation should improve not just accuracy but also how confidently and reliably your model makes predictions. So keep an eye on validation curves, calibration metrics and not just the final test accuracy. The key takeaway here is regularisation isn't one size fits all.

Tailor it to your use case and always test its real effect. Before we conclude here are four key tips to keep in mind when putting regularisation into practise. Start simple.

Add one technique at a time. Start with the simpler techniques also that you have learned in the other section. Simple regularisation techniques.

This helps you to isolate its effect and understand what's actually helping. Stacking everything at once might give results but you won't know what worked. Then hyperparameter search.

Many regularisation methods like dropout rate, label smoothing, epsilon or mix-up alpha need tuning. Don't expect great results with default values and be mindful of computational cost. Some methods like mix-up and drop connect can slow down training.

Always evaluate whether the performance gain justifies the time or resource trade-off and then use framework support. The good news is most libraries like pytorch, keras and tensorflow already have implementations. You don't need to reinvent the wheel.

Just configure wisely. So that's your advanced regularisation techniques. As I mentioned some of them will be implementing.

Some of them is purely for your knowledge purpose. Thank you.